

Agent anatomy

An agent is the reusable protocol unit in UVM, it groups the pieces that talk to one interface

Sequencer, driver, monitor, active vs passive

- The sequencer is where sequences meet hardware, it hands sequence items to the driver
- The driver is the active path
- The monitor is the observe path
- An active agent builds sequencer plus driver plus monitor, you can stimulate and watch
- A passive agent keeps the monitor only
- ConfigDB often sets is_active and the virtual interface on the agent path

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Agent anatomy

INTERACTIVE TOOL

Agent anatomy

Open the usual agent box: **sequencer** • **driver** • **monitor** (plus active vs passive).
Starter: active UART agent with all three present and the driver selected.

Starter example: active agent with sequencer, driver, and monitor — driver selected on the drive path.

Load starter example

Challenges 0 / 22 cleared

Quiz: bundle: An agent usually contains...

☐ sequencer, driver, and monitor for one protocol

☐ only the scoreboard

☐ only the DUT RTL

☐ Makefile targets

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

Bundle

Agent groups sequencer, driver, and monitor for one protocol.

Active

Builds drive path (sequencer + driver) plus monitor.

Passive

Monitor only — observe without driving.

vif

Shared handle: driver drives, monitor samples.

Lab

AGENT PRESET

IS_ACTIVE

active + driver

UVM_ACTIVE

Load preset

Demo passive

Explain

Reset

UVM agent lab starter

learn_uvm2017 — uvm agent

Real UVM literacy

```
wsl - module07-uvm-agent/examples/agent-sketch

# Track A - real Unix
# real shell session (Track A - UVM agent anatomy)

$ pwd
/mnt/d/proj/designs/learning/courses/Learn_uvm2017/module07-uvm-agent

$ ls examples/agent-sketch/
anatomy.txt

$ cat examples/agent-sketch/anatomy.txt
UVM agent anatomy sketch (literacy)

agent (uvm_agent) - one protocol interface
├─ sequencer (active only) + sequences / seq_items
├─ driver (active only) + virtual interface → DUT
└─ monitor (always) + analysis port → scoreboard/subscribers

Active (UVM_ACTIVE):
build sequencer + driver + monitor
stimulus: sequence → sequencer → driver → vif → DUT
observe: DUT → vif → monitor → analysis

Passive (UVM_PASSIVE):
build monitor only
observe only - no sequencer/driver drive path

ConfigDB (typical on agent path):
is_active - active vs passive mode
vif - shared handle for driver and monitor

One agent per protocol:
uart_agent, spi_agent, mem_agent - do not mix unrelated protocols

Connect phase (offline):
sequencer.seq_item_port ← driver.seq_item_export
monitor.analysis_port → scoreboard / coverage

Common mistakes:
two protocols in one agent
passive agent with no monitor (wrong - passive still monitors)
driver/monitor without vif set on agent path

$ grep -n "uvm_agent\|ArchAgent" ../../Learn_uvm2017_sv_verilator/module6/examples/architecture/architecture.sv | head
-8
15: * └─ ArchAgent (uvm_agent)
354: * ArchAgent: Agent for architecture example
370: class ArchAgent extends uvm_agent;
381:   `uvm_component_utils(ArchAgent)
473:   ArchAgent agent; // Agent: Contains sequencer, driver, monitor
502:   agent = ArchAgent::type_id::create("agent", this);
```

Real shell — agent anatomy sketch

Pitfalls to watch

- Do not put two unrelated protocols in one agent, split agents instead
- Do not assume passive means no monitor, it means no driver path
- Do not forget the virtual interface on the agent path before build ends
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, toggle active versus passive and explain what disappears
- On real UVM, sketch one active agent with labeled sequencer, driver, and monitor
- When you are ready, take the short quiz, then continue to TLM ports in the next module